

Product and Category management system

Scenario

A Product and Category Management System helps businesses manage products by organizing them into different categories.

It allows administrators to add, update, delete, and view product and category details.

The system makes product management simple, organized, and efficient.

DESIGN PRODUCT AND CATEGORY TABLES

Category and product table:

```
SQL> CREATE TABLE Category(Category_ID VARCHAR(10),Category_Name VARCHAR(50),Description VARCHAR2(50));  
Table created.  
  
SQL> CREATE TABLE Product(Product_ID VARCHAR(10),Product_Name VARCHAR2(20),Category_ID VARCHAR2(7),  
)  
Table created.
```

Define relationship and constraints relationships

The Category and Product tables are related through the Category_ID column. A category can have many products, so the relationship is one-to-many. Category_ID is the Primary Key in the Category table and the Foreign Key in the Product table. Constraints are used to maintain the accuracy and integrity of the data in both tables.

Primary Key (PK)

A Primary Key uniquely identifies each record in a table. In the Category table, Category_ID is the Primary Key, while in the Product table, Product_ID is the Primary Key.

Foreign Key (FK)

A Foreign Key connects two tables and maintains a relationship between them. In the Product table, Category_ID is the Foreign Key that refers to Category_ID in the Category table.

NOT NULL

The NOT NULL constraint ensures that a column cannot contain an empty value. In the Product table, Product_ID and Product_Name are defined as NOT NULL, so every product must have an ID and a name.

CHECK

The CHECK constraint ensures that the values entered satisfy a specified condition. In the Product table, Price can be checked to ensure it is greater than 0, and Stock_Quantity can be checked to ensure it is 0 or greater.

UNIQUE

The UNIQUE constraint prevents duplicate values in a column. In the Category table, Category_Name can be defined as UNIQUE so that the same category name cannot be entered more than once.

Defining relationship and creating constraints for product and category:

Primary key:

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_Category PRIMARY KEY (Category_ID);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT PK_Product PRIMARY KEY (Product_ID);  
Table altered.
```

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);  
Table altered.
```

Adding constraints:

NOT NULL:

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.  
  
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (Price > 0);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);  
Table altered.
```

INSERT VALUES:

Category Table:

```
SQL> INSERT INTO Category VALUES ('Cat0001', 'Electronics', 'Electronic gadgets and devices');  
1 row created.  
SQL> INSERT INTO Category VALUES ('Cat0002', 'Clothing', 'Men, Women and Kids Apparels');  
1 row created.  
SQL> INSERT INTO Category VALUES ('Cat0003', 'Footwear', 'Shoes, Sandals and Slippers');  
1 row created.  
SQL> INSERT INTO Category VALUES ('Cat0004', 'Home Appliances', 'Appliances used for household purposes');  
1 row created.  
SQL> INSERT INTO Category VALUES ('Cat0005', 'Books', 'Educational, Fiction and Non-Fiction Books');  
1 row created.  
SQL> INSERT INTO Category VALUES ('Cat0006', 'Beauty', 'Beauty and Personal Care Products');  
1 row created.
```

Product Table:

```
SQL> INSERT INTO Product VALUES ('P1001', 'Laptops', 'Cat0001', 67500, 2);  
1 row created.  
SQL> INSERT INTO Product VALUES ('P1002', 'Mobile', 'Cat0001', 25000, 10);  
1 row created.  
SQL> INSERT INTO Product VALUES ('P1003', 'TShirt', 'Cat0002', 800, 20);  
1 row created.  
SQL> INSERT INTO Product VALUES ('P1004', 'Jeans', 'Cat0002', 1500, 15);  
1 row created.  
SQL> INSERT INTO Product VALUES ('P1005', 'Home Appliances', 'Cat0003', 5000, 8);  
1 row created.  
SQL> INSERT INTO Product VALUES ('P1006', 'Books', 'Cat0004', 500, 30);  
1 row created.
```

After Insertion Category Table:

READ:

READ is used to retrieve and view data from a table.

The **SELECT** command is used to read the required records.

```
1 row created.

SQL> SELECT * FROM Category;

CATEGORY_I
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
Cat0001
Electronics
Electronic gadgets and devices

Cat0002
Clothing
Men, Women and Kids Apparels

CATEGORY_I
-----
CATEGORY_NAME
-----
DESCRIPTION
-----

Cat0003
Footwear
Shoes, Sandals and Slippers
```

```
Cat0004
Home Appliances

CATEGORY_I
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
Appliances used for household purposes

Cat0005
Books
Educational, Fiction and Non-Fiction Books

Cat0006

CATEGORY_I
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
Beauty
Beauty and Personal Care Products

6 rows selected.
```

Product Table:

```
SQL> SELECT * FROM Product;

PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGOR      PRICE STOCK_QUANTITY
-----
P1001
Laptops
Cat0001      67500          2

P1002
Mobile
Cat0001      25000         10
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGOR      PRICE STOCK_QUANTITY
-----

P1003
TShirt
Cat0002      800           20

P1004
Jeans
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGOR      PRICE STOCK_QUANTITY
-----
Cat0002      1500          15

P1005
Home Appliances
Cat0003      5000           8

P1006
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
CATEGOR      PRICE STOCK_QUANTITY
-----
Books
Cat0004      500           30

6 rows selected.
```

UPDATE:

UPDATE is used to modify existing records in a table. It changes specific column values based on a given condition.

After update , select the table and check it:

Category Table:

```
SQL> UPDATE Category SET Category_Name = 'Gadgets' WHERE Category_ID = 'Cat0001';

1 row updated.

SQL> SELECT * FROM Category;
```

CATEGOR	CATEGORY_NAME	DESCRIPTION
Cat0001	Gadgets	Electronic gadgets and devices
Cat0002	Clothing	Men, Women and Kids Apparels
Cat0003	Footwear	Shoes, Sandals and Slippers
Cat0004	Home Appliances	Appliances used for household purposes
Cat0005	Books	Educational, Fiction and Non-Fiction Books
Cat0006	Beauty	Beauty and Personal Care Products

```
6 rows selected.
```

Product Table:

```
SQL> UPDATE Product SET Price = 70000 WHERE Product_ID = 'P1001';

1 row updated.

SQL> SELECT * FROM Product;
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P1001	Laptops	Cat0001	70000	2
P1002	Mobile	Cat0001	25000	10
P1003	TShirt	Cat0002	800	20
P1004	Jeans	Cat0002	1500	15
P1005	Home Appliances	Cat0003	5000	8
P1006	Books	Cat0004	500	30

```
6 rows selected.
```

DELETE:

DELETE is used to remove existing records from a table.

It deletes selected rows based on a specified condition.

After deleting ,select the table and check it:

Category Table:

```
SQL> DELETE FROM Category WHERE Category_ID = 'Cat0006';
```

```
1 row deleted.
```

```
SQL> SELECT * FROM Category;
```

CATEGOR	CATEGORY_NAME	DESCRIPTION
Cat0001	Gadgets	Electronic gadgets and devices
Cat0002	Clothing	Men, Women and Kids Apparels
Cat0003	Footwear	Shoes, Sandals and Slippers
Cat0004	Home Appliances	Appliances used for household purposes
Cat0005	Books	Educational, Fiction and Non-Fiction Books

Product Table:

```
SQL> DELETE FROM Product WHERE Product_ID = 'P1006';
```

```
0 rows deleted.
```

```
SQL> SELECT * FROM Product;
```

PRODUCT_ID	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P1001	Laptops	Cat0001	70000	2
P1002	Mobile	Cat0001	25000	10
P1003	TShirt	Cat0002	800	20
P1004	Jeans	Cat0002	1500	15
P1005	Home Appliances	Cat0003	5000	8